Laboratorio N°3

Unidad Curricular: Tecnologías de Microprocesamiento

12/10/2026

Rosella Merello
Universidad Tecnológica del Uruguay
Paysandú, Uruguay
rosella.morello@estudiantes.utec.edu.uy

Diego Vázquez
Universidad Tecnológica del Uruguay
Paysandú, Uruguay
diego.vazquez@utec.edu.uy

Luis Nuñez
Universidad Tecnológica del Uruguay
Fray Bentos, Uruguay
luis.nunez@estudiantes.utec.edu.uy

***Resumen*—** En este trabajo se desarrollan y validan dos aplicaciones embebidas basadas en el microcontrolador ATmega328P programado en lenguaje C. El Problema C implementa un clasificador de color mediante fotocelda, conversión ADC, servomotor, pantalla LCD, comunicación UART y una tira WS2812B como indicador visual. Se realizó una calibración principal con iluminación blanca y una medición complementaria secuencial R-G-B para obtener patrones de referencia de las muestras. El Problema D implementa un piano electrónico de ocho notas con dos canales de audio, tres canciones seleccionables mediante UART y visualización en LCD. Ambos sistemas se organizaron mediante máquinas de estados y fueron verificados mediante simulación, calibración experimental y pruebas funcionales.

***Palabras claves*—**ATmega328P, ADC, sistemas embebidos, máquina de estados, UART, WS2812B.

I. INTRODUCCIÓN

El ATmega328P integra periféricos que permiten resolver aplicaciones de adquisición, control y comunicación. Entre sus recursos se encuentran el conversor analógico-digital, temporizadores, salidas PWM, interrupciones y módulos de comunicación serial. En este laboratorio se profundiza su programación en lenguaje C, utilizando registros del microcontrolador y funciones específicas para integrar sensores y actuadores. [2]

La metodología de trabajo consiste en probar cada bloque de manera independiente, integrar progresivamente los periféricos y verificar la solución mediante simulación y montaje físico. Este enfoque facilita la identificación de fallas de conexionado, temporización y configuración de registros antes de realizar la validación final.

II. OBJETIVOS

A. Objetivo general

Desarrollar e implementar aplicaciones embebidas utilizando el microcontrolador ATmega328P programado en lenguaje C, integrando sensores, actuadores, sistemas de comunicación y periféricos para resolver problemas de adquisición, procesamiento y control.

B. Objetivos específicos

- Configurar y utilizar los periféricos internos del ATmega328P, incluyendo ADC, temporizadores, PWM, UART e I²C.
- Integrar sensores y actuadores para realizar mediciones, procesar información y ejecutar acciones automáticas de control.
- Implementar interfaces de visualización y comunicación mediante LCD 16×2 y comunicación serial.
- Aplicar máquinas de estados y temporizaciones para organizar el comportamiento de los sistemas.
- Verificar las soluciones mediante simulación, montaje físico, registro de datos y evidencias audiovisuales.

III. MATERIALES

TABLE I. MATERIALES PRINCIPALES

Material / herramienta	Aplicación en el laboratorio
ATmega328P / Arduino UNO	Microcontrolador utilizado en las implementaciones.
LDR / fotocelda + resistencia de 10 kΩ	Adquisición de la respuesta óptica mediante ADC0.
LCD 16×2 + PCF8574	Visualización mediante interfaz I²C.
Servomotor	Posicionamiento de la bandeja de clasificación.
WS2812B	Indicador visual del color clasificado y validación del protocolo temporizado.
2 buzzers pasivos + 2N2222	Melodía y acompañamiento del piano electrónico.
8 pulsadores	Teclado de las ocho notas.
Resistencias, protoboard y cables jumper	Polarización, conexión y montaje de los circuitos.
Proteus 8 Professional	Simulación y verificación de los Problemas C y D.
Microchip/Atmel Studio	Desarrollo y compilación del código en C.
LEDs blancos	Iluminación fija para calibración principal por reflectancia.
LED RGB cátodo común + resistencias	Iluminación secuencial R-G-B para medición complementaria por canal.

IV. MARCO TEÓRICO

A. Microcontrolador ATmega328P

El ATmega328P es un microcontrolador AVR de 8 bits con memoria Flash, SRAM y EEPROM, puertos digitales configurables, conversor ADC, temporizadores y módulos de comunicación. En este laboratorio actúa como unidad central de adquisición, procesamiento y control.

B. Lenguaje C y registros AVR

El lenguaje C permite estructurar el programa en funciones de inicialización, adquisición y control, manteniendo acceso

directo a los registros del microcontrolador. La configuración de DDRx, PORTx y PINx determina el uso de los pines, mientras que registros específicos controlan ADC, temporizadores, PWM y comunicaciones.

C. Conversión analógico-digital

El ADC transforma una tensión analógica en un valor digital. En el Problema C se utiliza ADC0 para adquirir la señal de la fotocelda. En la versión con iluminación blanca se promedian diez conversiones y se compara el resultado con referencias calibradas. Como validación complementaria se realizan tres adquisiciones independientes bajo iluminación roja, verde y azul, generando un vector (ADC_R, ADC_G, ADC_B) para cada muestra. Esta segunda estrategia aporta mayor información cromática y permite utilizar un criterio de distancia mínima entre la medición y los patrones de referencia.

D. Comunicación UART

La UART permite transmitir y recibir información de forma serial mediante las líneas TX y RX. En el Problema C se emplea para reportar lecturas, clasificación y ángulo del servomotor; en el Problema D permite seleccionar canciones, detener la reproducción y retornar al modo piano.

E. Comunicación I²C y LCD

I²C utiliza las líneas SDA y SCL para comunicar el ATmega328P con dispositivos periféricos. El LCD 16×2 se controla mediante un expansor PCF8574 con dirección 0x27, reduciendo la cantidad de pines necesarios. Las líneas SDA y SCL emplean resistencias pull-up.

F. PWM, temporizadores y actuadores

Los temporizadores permiten generar señales periódicas con alta precisión. En el Problema C, Timer1 se utiliza para obtener el período de 20 ms requerido por el servomotor (50 Hz). En el Problema D, Timer1 y Timer2 generan las frecuencias de los dos buzzers y una base de tiempo independiente administra duraciones, antirrebote y cambios de nota sin depender exclusivamente de retardos bloqueantes.

G. WS2812B y fotocelda

El WS2812B es un LED RGB direccionable controlado mediante una línea digital con temporización estricta; el orden de datos utilizado es G-R-B. En el sistema se emplea como indicador visual del color clasificado y su trama se genera mediante código de temporización precisa. Para estudiar la respuesta óptica por canal también se realizó una medición complementaria con un LED RGB de cátodo común, activando secuencialmente R, G y B y registrando la fotocelda en cada condición. Esta prueba reproduce el principio de adquisición cromática solicitado por la práctica y permite documentar patrones separados por canal.

H. Máquina de estados

Una máquina de estados finitos organiza el comportamiento del sistema en estados definidos y transiciones asociadas a entradas o eventos. Este enfoque facilita separar modos de operación, como piano y canción, y documentar de forma clara el flujo de cada aplicación.

V. METODOLOGÍA Y DESARROLLO

Antes de integrar los periféricos se verifica la alimentación de 5 V, la referencia común de GND, la asignación de pines y la frecuencia de reloj del ATmega328P. Se comprueba además la dirección I²C 0x27 del PCF8574, la entrada analógica A0 de la fotocelda y las salidas destinadas al servomotor y a la señal de la tira WS2812B. La integración se realiza de forma incremental: primero adquisición y visualización, luego clasificación y actuación, y finalmente simulación del sistema completo. Este procedimiento se adopta para mantener trazabilidad y reducir fallas de conexionado o configuración.

A. Problema A – Control de temperatura

El sistema mide la temperatura cada 5 s con un sensor DHT11 y la compara con un punto medio configurable (20 °C por defecto). Según la zona en la que se encuentra, enciende un calefactor o regula la velocidad de un ventilador mediante PWM. La temperatura y la acción ejecutada se informan por UART y en un LCD 16×2 con interfaz I²C, y los datos se registran para graficarlos con Python.

1) Designación de pines

TABLE II. DESIGNACIÓN DE PINES – PROBLEMA A

Función	Arduino	ATmega328P
DHT11 (DATA)	D4	PD4
Calefactor	D2	PD2
Ventilador (PWM)	D9	PB1 / OC1A
LCD SDA	A4	PC4
LCD SCL	A5	PC5
UART RX/TX	D0 / D1	PD0 / PD1

2) Lógica de control

Los umbrales se expresan como desplazamientos respecto del punto medio M, de modo que al modificar M todos los rangos se recalculan automáticamente. Con M = 20 °C se obtienen los rangos del enunciado.

TABLE III. ZONAS DE CONTRAL – PROBLEMA A

Zona	Condiciòn	Con M = 20 °C	Calefactor	Ventilador
Frìa	$T \le M-5$	$T \le 15$	ON	0 %
Neutra	$M-5 < T \le M+5$	$15 < T \le 25$	OFF	0 %
Tibis	$M+5 < T \le M+15$	$25 < T \le 35$	OFF	40 %
Caliente	$M+15 < T \le M+25$	$35 < T \le 45$	OFF	70 %
Muy Caliente	T > M + 25	T > 45	OFF	100 %

3) Funcionamiento Integrado

La base de tiempo la genera el Timer0 en modo CTC con una interrupción cada 1 ms; al cumplirse 5000 ms se

levanta una bandera que el programa principal atiende para medir. De este modo no hay esperas bloqueantes y la UART se atiende en todo momento. El ventilador se maneja con el Timer1 en modo Fast PWM a 1 kHz, con ciclos de trabajo de 40 %, 70 % y 100 %.. El calefactor es una salida digital [representada con un LED en la simulación].

El punto medio se modifica desde un menú por UART (tecla m). El valor debe ser un entero entre 5 y 45 °C; fuera de ese rango el sistema notifica y no realiza el cambio. Al aceptar un valor, los umbrales se recalculan automáticamente y el LCD muestra la temperatura y el punto medio con su rango neutro. Si el sensor falla en tres lecturas consecutivas, el calefactor y el ventilador se apagan por seguridad.

Cada medición se informa con una línea legible y una línea de datos con formato fijo (D, muestra, temperatura, punto medio, acción, PWM). Un script en Python lee esas líneas, en vivo o desde un registro, las guarda en CSV y grafica la evolución de la temperatura con la banda del rango ideal, el estado del calefactor y el PWM del ventilador.

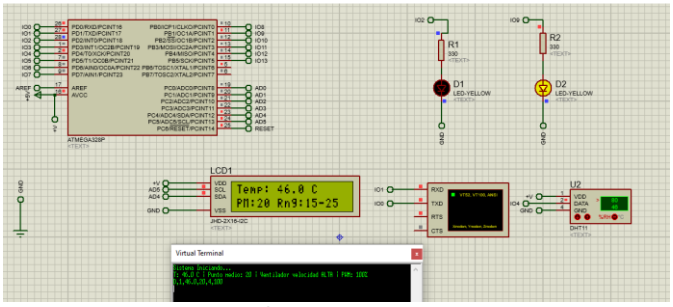


Fig. 1. Esquema de simulación del Problema A en Proteus: ATmega328P, DHT11, LED del calefactor, salida PWM del ventilador, LCD I²C y terminal virtual UART.

3) Registro y gráfica de datos

Cada medición válida se transmite como una línea de datos de formato fijo: `D,muestra,temperatura,punto medio,acción,PWM`. El script de Python extrae estas líneas mediante una expresión regular, ya sea desde un registro de texto copiado de la terminal o en vivo desde un puerto serie, calcula el tiempo como (muestra − 1) × 5 s, guarda los datos en un archivo CSV y genera una figura de tres paneles: temperatura medida con la banda del rango ideal (punto medio ± 5 °C), estado del calefactor (ON/OFF) y velocidad del ventilador (% PWM). Como el tiempo se calcula a partir del número de muestra, las lecturas con error generan huecos visibles en el eje temporal en lugar de desplazar la gráfica.

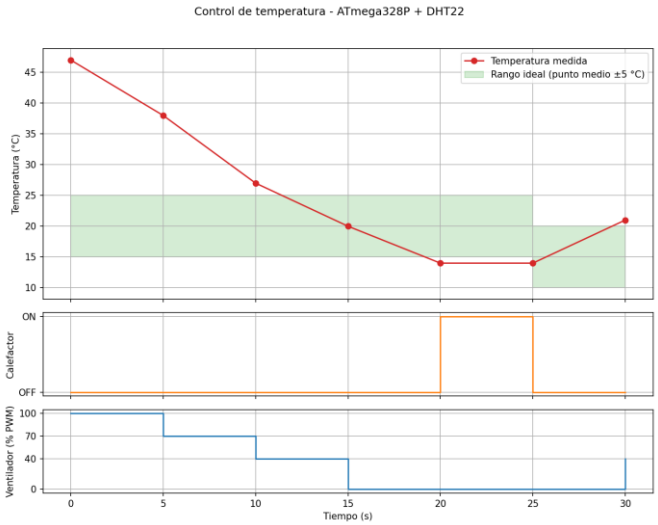


Fig. 2. Gráfica generada con Python: temperatura medida y rango ideal, estado del calefactor y velocidad del ventilador.

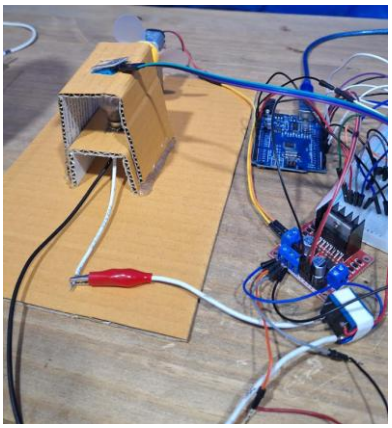


Fig. 3. Montaje físico presentado para el Ejercicio A.

B. Problema B – Control automático de cortina

El sistema regula la apertura de una cortina utilizando una LDR para medir la iluminación y un potenciómetro acoplado al eje como realimentación de posición. El ATmega328P recibe por UART un setpoint entre 0 y 1023; compara este valor con la lectura de luz y acciona un motor de corriente continua mediante un puente H L298N. La lectura de posición permite limitar el recorrido y detener el mecanismo cerca de sus extremos.

1) Asignación de señales

En la versión simulada, el potenciómetro de posición se lee por A0/ADC0 y la LDR por A1/ADC1. El control del L298N utiliza D9/PB1 (ENA, PWM), D8/PB0 (IN1) y D7/PD7 (IN2). La comunicación con el terminal virtual se realiza por UART en D0/PD0 (RX) y D1/PD1 (TX), a 9600

baudios. El L298N y el microcontrolador comparten GND; el motor se alimenta desde la fuente prevista para el controlador.

2) Lógica de control e integración

En cada ciclo se adquieren la iluminación y la posición. Una banda de histéresis evita invertir el motor ante pequeñas variaciones alrededor del setpoint. Si la lectura queda fuera de esa banda, el programa selecciona el sentido de giro con IN1 e IN2 y regula la velocidad mediante PWM; al alcanzar la condición de parada o el límite de recorrido, desactiva el motor. Por UART se muestran el setpoint (SP), la lectura LDR, la posición y la acción (ABRIR, CERRAR o PARADO), lo que permite seguir el comportamiento durante la prueba.

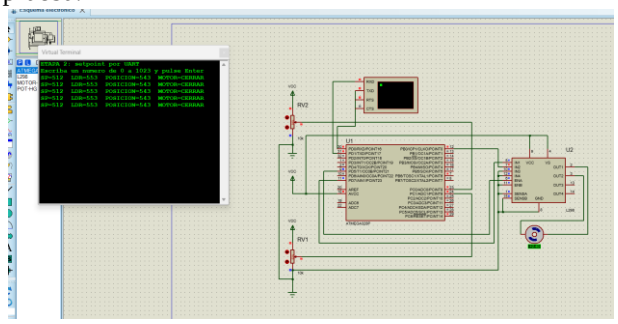


Fig. 4. Simulación del Problema B en Proteus: ATmega328P, entradas analógicas, terminal UART, L298N y motor de la cortina.

3) Registro de datos y gráfica en Python

Para registrar una prueba se conecta la placa al computador, se selecciona el puerto COM correcto y se cierra el Monitor Serie del IDE, porque el puerto no puede ser utilizado simultáneamente por dos programas. El script graficar_cortina.py abre la comunicación a 9600 baudios, solicita el setpoint, lo envía al controlador y registra durante 30 s las líneas recibidas. En la prueba documentada se usó SP = 180. Al finalizar se guarda datos_cortina.csv y se genera una gráfica SVG. Para volver a dibujar los datos guardados se ejecuta recrear_grafica.py; esta etapa puede realizarse sin tener la cortina conectada, porque lee el CSV ya creado. En la gráfica, el rojo representa el setpoint, el azul la lectura LDR el verde la posición del potenciómetro. Las lecturas corresponden a cuentas ADC, no a lux ni a grados mecánicos.

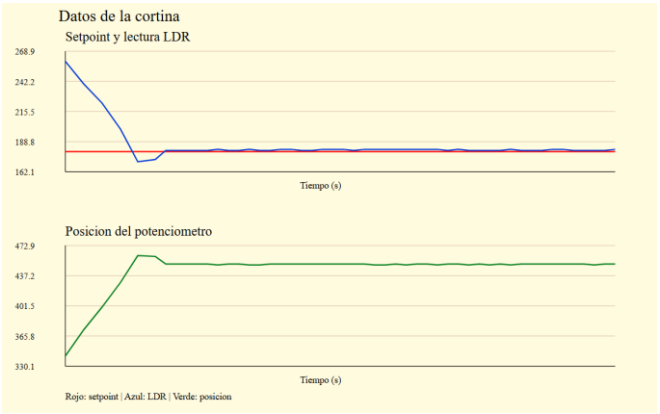


Fig. 5. Registro de 30 s generado con Python para SP = 180: setpoint, lectura LDR y posición del potenciómetro.

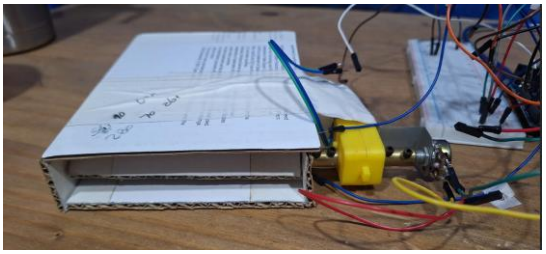


Fig. 6. Montaje físico de la cortina del Problema B.

C. Problema C – Clasificador de color de plásticos

El Problema C se desarrolló en dos variantes complementarias. La primera utiliza iluminación blanca fija para obtener una lectura ADC estable y clasificar mediante umbrales calibrados; una vez determinado el color, la tira WS2812B se enciende con el color reconocido y el servomotor se posiciona en el ángulo correspondiente. La segunda variante utiliza un LED RGB convencional como fuente de iluminación secuencial R-G-B y registra tres lecturas independientes de la fotocelda. Esta medición adicional permite construir patrones cromáticos y aplicar una comparación por distancia mínima, aproximándose al método por canales requerido en la consigna.

1) Designación de pines

TABLE IV. DESIGNACIÓN DE PINES – PROBLEMA C

Función	Arduino	ATmega328P
Fotocelda	A0	PC0 / ADC0
WS2812B DIN	D6	PD6
Servomotor	D9	PB1 / OC1A
LCD SDA	A4	PC4
LCD SCL	A5	PC5
UART RX/TX	D0 / D1	PD0 / PD1
LED RGB rojo (calibración)	D3	PD3
LED RGB verde (calibración)	D4	PD4

Función	Arduino	ATmega328P
LED RGB azul (calibración)	D5	PD5

2) Calibración y patrones de referencia

La calibración principal se realizó con el montaje definitivo, manteniendo constantes la posición de la muestra, la fotocelda y la iluminación blanca. Se observaron lecturas altamente estables, con variaciones de uno o dos conteos como máximo. Los límites de clasificación se obtuvieron mediante el punto medio entre referencias consecutivas: 555, 598, 646 y 705. Esta elección deja un margen simétrico entre clases y evita utilizar límites arbitrarios. Debido a que las muestras empleadas son opacas, la magnitud observada físicamente corresponde principalmente a reflectancia; se conserva la fotocelda como elemento de adquisición exigido por la práctica.

TABLE V. CALIBRACIÓN PRINCIPAL CON ILUMINACIÓN BLANCA – PROBLEMA C

Condición	ADC ref.	Rango	WS2812	Servo
Sin objeto	542	0–555	Apagado	90°
Violeta	568	556–598	75, 0, 130	115°
Verde manzana	628	599–646	124, 252, 0	25°
Naranja	665	647–705	255, 69, 0	65°
Blanco	746	706–1023	255, 255, 255	155°

TABLE VI. MEDICIÓN COMPLEMENTARIA R-G-B – PROBLEMA C

Condición	ADC_R	ADC_G	ADC_B
Sin objeto	79	96	55
Violeta	97	115	159
Verde manzana	133	259	84
Naranja	318	138	100
Blanco	331	373	293

Como medición complementaria se activaron sucesivamente los canales rojo, verde y azul de un LED RGB de cátodo común conectado mediante resistencias limitadoras. Para cada canal se promediaron lecturas del ADC y se obtuvo una firma (R,G,B). La clasificación de esta variante se implementó con distancia Manhattan: Dk = |R-Rk| + |G-Gk| + |B-Bk|, seleccionando el patrón con menor Dk. El criterio evita operaciones en punto flotante, es adecuado para el ATmega328P y satisface la necesidad de justificar una regla de decisión basada en referencias calibradas.

3) Funcionamiento integrado

En la variante principal, el programa promedia diez lecturas de ADC0, determina la clase según los umbrales calibrados, calcula la diferencia respecto a la referencia, actualiza la tira WS2812B y posiciona el servomotor. Timer1 trabaja en Fast PWM con período de 20 ms, equivalente a 50 Hz. El LCD muestra ADC, color y ángulo, mientras UART reporta ADC, color, referencia, diferencia y posición del servo. En la variante RGB, cada canal se ilumina y mide por separado antes de aplicar la distancia mínima. La separación entre adquisición, decisión y actuación facilita la validación por estados y la comparación con la rúbrica.

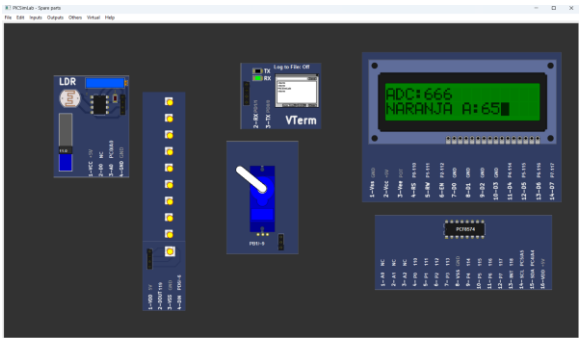


Fig. 7. Simulación del Problema C en PICSimLab. Para ADC = 666 el sistema clasifica NARANJA y posiciona el servomotor en 65°.

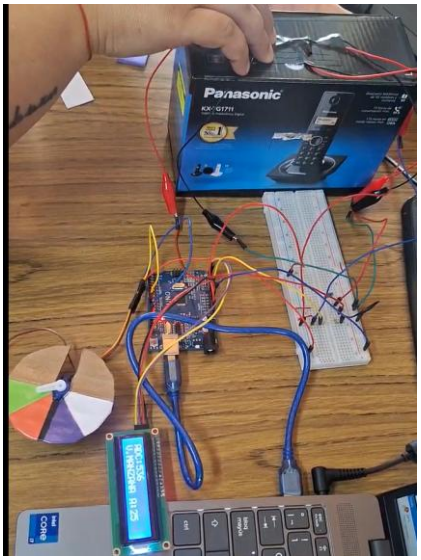


Fig. 8. Montaje físico utilizado durante las pruebas del Problema C.

D. Problema D – Piano electrónico

El Problema D opera en modo piano y modo canción. Ocho pulsadores representan las notas DO, RE, MI, FA, SOL, LA, SI y DO agudo. Las salidas de audio se asignan a dos canales independientes para permitir melodía y acompañamiento, y la arquitectura se organiza mediante estados de inicialización, espera/modo piano, nota activa y canción activa. La LCD presenta el modo o canción actual y la UART permite seleccionar repertorio y retornar al modo piano.

1) Designación de pines

TABLE VII. DESIGNACIÓN DE PINES – PROBLEMA D

Función	Arduino	ATmega328P
DO	D2	PD2
RE	D3	PD3
MI	D4	PD4
FA	D5	PD5
SOL	D6	PD6
LA	D7	PD7
SI	A2	PC2
DO agudo	A3	PC3
BUZ1 melodía	D9	PB1 / OC1A

Función	Arduino	ATmega328P
BUZ2 acompañamiento	D11	PB3 / OC2A
LCD SDA/SCL	A4/A5	PC4/PC5
UART RX/TX	D0/D1	PD0/PD1

2) Generación de notas

Las notas se generan a partir de temporizadores configurados para producir frecuencias discretas. Timer1 se asigna a la melodía y Timer2 al acompañamiento, de modo que el diseño admite dos frecuencias simultáneas. Los ocho pulsadores se leen con lógica activa en bajo y resistencias pull-up. Para evitar múltiples detecciones por una sola pulsación, la implementación final debe conservar antirrebote y lectura estable de cada tecla, tal como exige la rúbrica.

TABLE VIII. FRECUENCIAS DEL PIANO

Nota	Frecuencia
Do	262 Hz
Re	294 Hz
Mi	330 Hz
Fa	349 Hz
Sol	392 Hz
La	440 Hz
Si	494 Hz
Do agudo	523 Hz

3) Selección de canciones por UART

La interfaz de entrega se define con los comandos C1, C2 y C3 para seleccionar Imagine, New York, New York y A Whole New World, respectivamente, y STOP para detener la reproducción y regresar al modo piano. Durante la depuración en Proteus se utilizaron también alias numéricos 1, 2 y 3 y la tecla P para Piano/Stop; estos alias se observan en la evidencia de simulación y permiten agilizar las pruebas sin modificar la correspondencia de las tres canciones.

TABLE IX. COMANDOS UART – PROBLEMA D

Comando	Función
C1	Reproducir Imagine
C2	Reproducir New York, New York
C3	Reproducir A Whole New World
STOP	Detener ambos buzzers y volver al modo piano
P	Alias opcional de STOP

4) Temporización, polifonía y antirrebote

Para alcanzar respuesta simultánea, la generación de tono se asocia a Timer1 y Timer2, mientras la lógica de estados determina cuándo iniciar, mantener o detener cada canal. Las duraciones y cambios de nota deben depender de eventos temporizados y no exclusivamente de retardos bloqueantes. El antirrebote de los pulsadores se integra en el estado de lectura del teclado. Esta separación permite que UART, LCD y teclado continúen siendo atendidos mientras una canción se encuentra activa.

5) Simulación y verificación

La simulación se realizó en Proteus 8 Professional con el ATmega328P, ocho pulsadores, dos salidas de audio, LCD 16×2, PCF8574 y terminal virtual UART. En la evidencia final se observan las tres canciones ejecutadas —Imagine, New York, New York y A Whole New World— y el retorno a MODO PIANO / ESPERANDO. Esta prueba valida la navegación entre modos, la visualización y la selección del repertorio. Como criterio de cierre de nivel Excelente, la demostración audiovisual debe mostrar además las dos señales de audio simultáneas y la implementación con 2N2222.

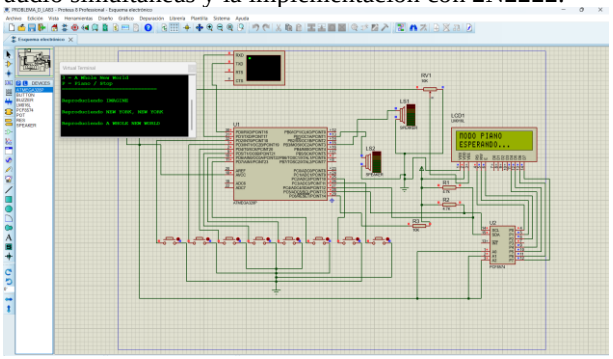


Fig. 9. Simulación del Problema D en Proteus: selección de las tres canciones por UART y retorno al modo piano con información en LCD.

VI. RESULTADOS Y ANÁLISIS

A. Problema A

En simulación con Proteus (ATmega328P a 16 MHz y UART a 9600 baud) se verificó el correcto funcionamiento del sistema: la lectura del DHT11, el cambio de zona al variar la temperatura del sensor, el PWM de 40 %, 70 % y 100 % en el osciloscopio virtual, el rechazo de valores de punto medio fuera de rango o no numéricos, la actualización del LCD y el apagado de los actuadores tras tres errores consecutivos del sensor. La gráfica generada con Python (Fig. 2) muestra una adecuada respuesta ante las variaciones de temperatura, estabilidad y desplazamiento de la banda del rango ideal al cambiar el punto medio.

B. Problema B

En la simulación de Proteus se observó la respuesta del controlador al setpoint recibido por UART; una lectura registrada muestra SP = 512, LDR = 553, posición = 543 y acción CERRAR, coherente con una lectura de luz superior al valor objetivo. En el registro con Python se aplicó SP = 180 durante 30 s. La gráfica muestra que la lectura LDR parte de un valor mayor, se aproxima al setpoint y luego permanece cerca de él; la posición aumenta durante la apertura y se estabiliza al final del movimiento. Se aprecia un transitorio inicial y pequeñas variaciones en régimen estable. El registro permite evaluar la respuesta y orientar el ajuste de la histéresis, la velocidad PWM y la duración de los pulsos para reducir la búsqueda sin perder resolución.

C. Problema C

La calibración con iluminación blanca produjo referencias de 542, 568, 628, 665 y 746 para sin objeto, violeta, verde manzana, naranja y blanco, respectivamente. La separación mínima entre referencias consecutivas fue de 26 cuentas ADC, mientras que las mediciones repetidas mostraron variaciones

muy pequeñas; esto permitió establecer umbrales por punto medio con margen claro entre clases. En PICSimLab se verificó el caso ADC = 666, incluido dentro del intervalo 647–705, obteniéndose correctamente NARANJA y un ángulo de 65°. La prueba complementaria R-G-B mostró firmas coherentes con el color de cada muestra: violeta presentó mayor respuesta azul, verde manzana mayor respuesta verde, naranja mayor respuesta roja y blanco niveles altos en los tres canales. La geometría y la iluminación deben mantenerse constantes porque la fotocelda es sensible a luz ambiente, distancia y orientación.

D. Problema D

En Proteus se verificó el modo piano, la interfaz serial, la visualización LCD y la reproducción de las tres canciones. El terminal registra la ejecución de Imagine, New York, New York y A Whole New World, mientras la pantalla retorna a MODO PIANO / ESPERANDO después de las pruebas. El esquema incorpora dos salidas de audio y ocho pulsadores, lo que permite validar la arquitectura requerida. Para evidenciar completamente el nivel Excelente de la rúbrica, el video final debe mostrar las ocho notas con antirrebote, la actuación simultánea de melodía y bajo mediante dos canales y los comandos C1/C2/C3/STOP —o sus alias junto con estos comandos— durante una ejecución completa.

Análisis general

La comparación con la rúbrica muestra que la calidad de la evidencia depende tanto del resultado como del método.En cuanto al Problema A, Proteus compila el código sin optimización, por lo que _delay_ms() y _delay_us() no compilaron y se reemplazaron por retardos propios basados en _delay_loop_2(); además, el modelo del DHT22 no funcionó en la simulación, por lo que se utilizó el DHT11, y el LCD I²C simulado requirió el protocolo de su controlador (ST7032) en lugar del PCF8574. En el Problema C se documentaron calibración, criterio de decisión, PWM de 50 Hz, comunicación y dos estrategias de adquisición óptica. La versión RGB complementaria fortalece el criterio de medición por canales, mientras la simulación con WS2812B evidencia el control visual del color. En el Problema D, la FSM separa claramente teclado, canciones y retorno a espera; el uso de dos temporizadores permite justificar la polifonía y evita depender únicamente de retardos. La evidencia final debe ser legible y mostrar entradas, salidas y transiciones relevantes.

VII. IMPLEMENTACIÓN EN LENGUAJE C

Los programas de los Problemas C y D se implementan en lenguaje C para el ATmega328P, separando rutinas de inicialización, adquisición, temporización, comunicación y accionamiento. En el Problema C se emplean funciones para ADC, UART, LCD, servomotor y generación de la trama WS2812B. La calibración blanca utiliza umbrales por punto medio, mientras la variante RGB procesa tres canales y calcula distancia mínima. En el Problema D se separan lectura de teclado, generación de tonos, selección UART, actualización LCD y control de estados.

En el Problema C Timer1 mantiene el servomotor a 50 Hz y la rutina WS2812B utiliza temporización precisa con interrupciones deshabilitadas durante la transmisión de cada trama. En el Problema D Timer1 y Timer2 se reservan para las dos señales de audio, y la máquina de estados coordina teclado, canciones y retorno a espera. Esta organización evita acoplar toda la lógica al bucle principal y facilita la validación individual de cada subsistema.

VIII. MÁQUINAS DE ESTADOS

A. Problema A

Para el control de temperatura se definió una máquina de estados con inicialización, espera, medición, cinco estados de acción (calefactor y ventilador a velocidad baja, media y alta, más el estado neutro) y un estado de falla del sensor. Cada 5 s, la bandera del Timer0 lleva el sistema a la medición; con una lectura válida pasa al estado que corresponde a la temperatura respecto del punto medio y, con tres errores consecutivos, pasa a la falla, donde el calefactor y el ventilador quedan apagados hasta recuperar una lectura válida. Una segunda máquina organiza la interfaz por UART en tres estados: monitoreo, menú y espera de valor.

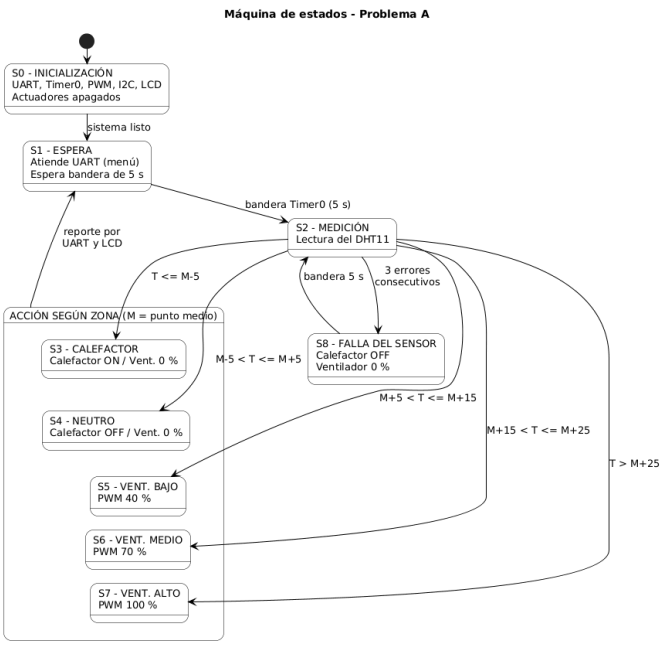


Fig. 10. Diagrama de estados del Problema A.

B. Problema B

La máquina de estados inicia con la configuración del ADC, la UART y el PWM, y mantiene el motor detenido. Luego recibe o actualiza el setpoint, lee la LDR y la posición, y evalúa si la diferencia respecto al setpoint supera la histéresis. Según la lectura y los límites de recorrido, ordena abrir, cerrar o detener. Después informa las variables por UART y vuelve a leer los sensores para corregir gradualmente la posición.

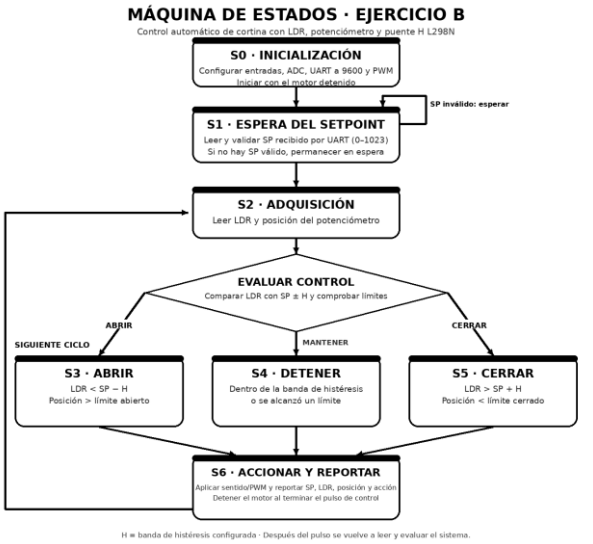


Fig. 11. Máquina de estados del Problema B para lectura del setpoint, histéresis y control de apertura/cierre.

C. Problema C

La máquina de estados del Problema C representa la variante de adquisición R-G-B. El ciclo comienza con la espera, continúa con iluminación y lectura independiente de los canales R, G y B, pasa a clasificación mediante patrones calibrados y finaliza con actuación y reporte. La variante de iluminación blanca utilizada para la calibración principal puede interpretarse como una simplificación de la misma FSM: medición ADC → clasificación → actuación. En ambos casos se mantienen separadas adquisición, decisión y salida.

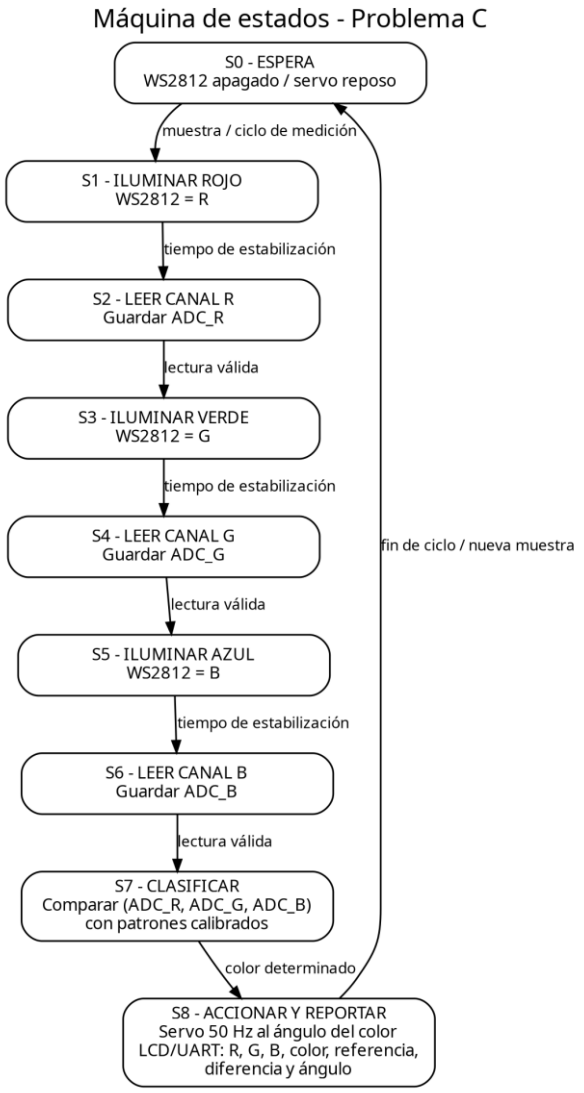


Fig. 12. Máquina de estados del Problema C para adquisición secuencial R-G-B, clasificación por patrones y actuación.

D. Problema D

La máquina de estados del Problema D contempla inicialización, modo piano/espera, nota activa y canción activa. Desde S1, una tecla estable lleva a S2; al liberarla se retorna a espera. Los comandos C1, C2 o C3 llevan a S3, donde la melodía y el bajo avanzan por eventos temporizados. STOP o el final de la canción devuelven el sistema a S1. Esta representación explicita estados, transiciones y condiciones de salida, tal como solicita la rúbrica.

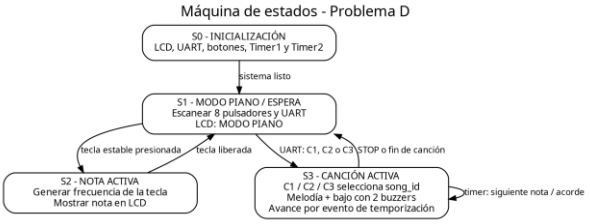


Fig. 13. Máquina de estados del Problema D con modo piano, nota activa, canción activa, C1/C2/C3 y STOP.

IX. CONCLUSIONES

En el Problema A se implementó un controlador de temperatura que integra un sensor digital de un solo hilo, un temporizador en modo CTC como base de tiempo, PWM por hardware, comunicación UART con menú interactivo y visualización en LCD I²C. Los umbrales se expresan como desplazamientos respecto del punto medio, lo que permite recalcular automáticamente todos los rangos al modificarlo, y la lógica de seguridad apaga los actuadores ante fallas repetidas del sensor. Durante la simulación fue necesario adaptar el diseño a las limitaciones del entorno: se utilizaron retardos propios en lugar de `util/delay.h`, el DHT11 en lugar del DHT22, cuyo modelo no funcionó, y el protocolo del controlador ST7032 para el LCD. El registro de datos y su gráfica con Python permitieron analizar la estabilidad y la respuesta del sistema. Los Problemas C y D integran adquisición analógica, comunicación UART e I²C, temporizadores y control de actuadores en el ATmega328P. En el clasificador se obtuvo una calibración estable con iluminación blanca y, adicionalmente, una firma R-G-B para cada muestra mediante iluminación secuencial. La comparación de ambas estrategias permitió justificar tanto el uso de umbrales por punto medio como un criterio de distancia mínima. La actuación combina confirmación visual mediante WS2812B, posicionamiento del servomotor a 50 Hz y reporte por LCD/UART.

El piano electrónico permitió organizar ocho entradas, dos canales de audio, tres canciones y una interfaz serial dentro de una máquina de estados. La simulación confirmó la selección del repertorio y el retorno al modo piano, mientras el diseño con Timer1 y Timer2 establece la base para polifonía simple y atención concurrente de teclado, LCD y UART. La FSM facilita justificar la lógica y documentar las transiciones exigidas por la rúbrica.

La revisión final frente a la rúbrica muestra que el nivel Excelente depende de conservar evidencias legibles de todas las etapas: simulación previa, verificación eléctrica, calibración real, temporización, criterio de clasificación, polifonía, comandos UART, FSM y trazabilidad en GitHub. El informe deja documentados estos elementos y define pruebas de aceptación concretas para que el video y el repositorio coincidan con la implementación presentada.

En el Problema B se integraron la medición de luz, la realimentación de posición y el accionamiento reversible del motor, junto con el monitoreo UART y el registro de datos en Python. La gráfica permitió distinguir el transitorio de aproximación del comportamiento estable y mostró la utilidad de ajustar la histéresis y la velocidad para limitar las oscilaciones alrededor del setpoint.

REFERENCIAS

[1] Universidad Tecnológica del Uruguay, "Laboratorio 3 - Tecnologías de Microprocesamiento," 2026.

[2] Microchip Technology Inc., "ATmega328P 8-bit AVR Microcontroller Datasheet."

[3] Worldsemi, "WS2812B Intelligent Control LED Integrated Light Source Datasheet."

[4] NXP Semiconductors, "PCF8574 Remote 8-bit I/O Expander for I²C-bus Datasheet."

Acceso a carpeta Drive:
https://drive.google.com/drive/folders/1aAokjXDrKJhH6K78yRNLe3dcJEi1j5ma?usp=sharing